

AI Backend Engineering Project – Complete Setup Guide (Phase 0 to Phase 4)

This document provides **end-to-end setup steps** for all four phases of the AI Backend Engineering project. By following this guide, you can set up the system from scratch and understand how each phase builds on the previous one.

Phase 0 – Engineering Foundations (Python OOP + Project Setup)

Objective

Establish a clean, modular Python project using Object-Oriented Programming principles and Git version control.

Prerequisites

- Python 3.9+
- Git
- VS Code (recommended)

Setup Steps

Create virtual environment

```
python -m venv venv  
source venv/bin/activate # Windows: venv\Scripts\activate
```

1.

Initialize Git repository

```
git init
```

2.

Create project structure

```
app/  
  domain/
```

user.py
document.py
core/
config.py

3.

4. **Apply OOP principles**

- Encapsulation (classes for User, Document)
- Single Responsibility Principle
- Clear method naming

Create .gitignore

venv/
__pycache__/
.env

5.

Outcome

✓ Clean Python project with OOP foundations and Git tracking

Phase 1 – Backend API Development (FastAPI + MySQL)

Objective

Build REST APIs for users and documents using FastAPI and SQLAlchemy ORM.

Additional Prerequisites

- MySQL (or PostgreSQL)
- Docker (optional but recommended)

Setup Steps

Install dependencies

pip install fastapi uvicorn sqlalchemy pymysql python-dotenv

1.

Database configuration

DATABASE_URL=mysql+pymysql://user:password@localhost/db_name

2.

Create SQLAlchemy models

```
app/models/  
user.py  
document.py
```

3.

Create CRUD layer

```
app/crud/  
user.py  
document.py
```

4.

Create API routes

```
app/api/routes/  
users.py  
documents.py
```

5.

Run server

```
uvicorn app.main:app --reload
```

6.

Outcome

✓ Functional REST API with persistent database

Phase 2 – OCR Document Ingestion

Objective

Enable document uploads and extract text from PDFs/images using OCR.

Additional Prerequisites

- Tesseract OCR
- Poppler (for PDFs)

Setup Steps

Install OCR dependencies

`pip install pytesseract pillow pdf2image`

- 1.
2. **Install system tools**
 - Windows: Install Tesseract & Poppler, add to PATH

Linux:

`sudo apt install tesseract-ocr poppler-utils`

○

OCR service implementation

`app/services/ocr.py`

- 3.

Upload endpoint

`POST /documents/upload`

- 4.
5. **Store extracted text in database**

Outcome

✓ Documents can be uploaded and converted to text

Phase 3 – Semantic Search (Chunking + Embeddings + FAISS)

Objective

Enable semantic search over documents using embeddings and vector similarity.

Additional Prerequisites

- FAISS
- Sentence Transformers

Setup Steps

Install dependencies

pip install faiss-cpu sentence-transformers

1.

Create document chunk table

DocumentChunk

- id
- document_id
- content

2.

Implement chunking logic

app/services/chunking.py

3.

4. Indexing pipeline (IMPORTANT)

- Chunk text
- Store chunks in DB
- Embed each chunk
- Store embeddings in FAISS

Semantic search function

search_chunks(query, k)

5.

Outcome

- ✓ Fast semantic search over document chunks

Phase 4 – Agentic AI with LangGraph + Azure OpenAI

Objective

Create an AI agent that can:

- Detect user intent
- Retrieve relevant chunks
- Decide between document vs web
- Answer or summarize

Additional Prerequisites

- Azure OpenAI account
- LangGraph

Environment Variables

```
AZURE_OPENAI_ENDPOINT=...  
AZURE_OPENAI_KEY=...  
AZURE_OPENAI_API_VERSION=...  
AZURE_OPENAI_DEPLOYMENT=...
```

Setup Steps

Install dependencies

```
pip install langgraph openai
```

1.

LLM client setup

```
app/ai/llm.py
```

2.

Define agent state

```
app/ai/state.py
```

3.

Create LangGraph nodes

```
intent.py    # intent classification  
retrieve.py  # semantic retrieval  
decide.py    # document vs web  
answer.py    # answer from docs  
summarize.py # summarization  
web.py       # web fallback
```

4.

Build LangGraph workflow

```
app/ai/graph.py
```

5.

Expose AI endpoint

```
POST /ai/ask
```

6.

Outcome

- ✓ Agentic AI system capable of reasoning and routing

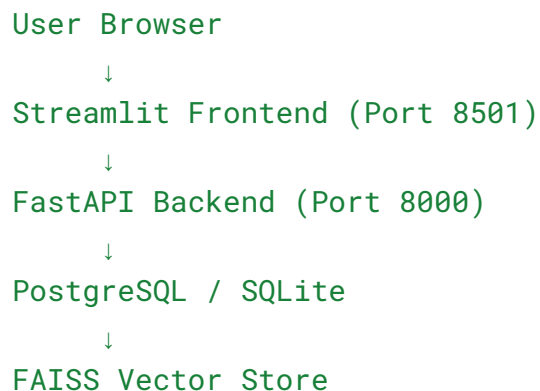
In **Phase 5**, we add a **user-friendly frontend** using **Streamlit** and **Dockerize the entire system**.

By the end of this phase, the project will:

- Have a clean **Streamlit UI**
 - Allow users to upload documents, view extracted text, and ask questions
 - Run **Backend + Frontend using Docker**
 - Follow **production-style separation** (thin frontend, heavy backend)
-

Phase 5:

Architecture Overview



Folder Structure (After Phase 5)

```
ai-backend/
|
├── app/                # FastAPI backend
```

```
|   |— api/
|   |— core/
|   |— crud/
|   |— models/
|   |— schemas/
|   |— services/
|   └─ main.py
|
|— streamlit_app/          # Streamlit frontend
|   |— app.py
|   |— Dockerfile.streamlit
|   |— requirements.streamlit.txt
|   └─ .dockerignore
|
|— Dockerfile.backend
|— docker-compose.yml
|— requirements.txt
└─ README.md
```

Features Implemented in Phase 5

Streamlit Frontend

- Upload PDF / Image documents
- View uploaded documents
- View extracted text per document
- Ask AI questions (RAG-based)
- Top-K chunk selection
- Clear UI using tabs

Dockerization

- Backend Docker image
 - Streamlit Docker image
 - Orchestrated using Docker Compose
 - Optimized build times
-

Prerequisites

Make sure the following are installed:

- **Python 3.10+**
- **Docker Desktop**
- **Docker Compose**
- **Git**

Verify:

```
docker --version
docker compose version
```

Backend Dockerfile (**Dockerfile.backend**)

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
RUN apt-get update && apt-get install -y \
    build-essential \
    poppler-utils \
    tesseract-ocr \
    && rm -rf /var/lib/apt/lists/*
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Streamlit Dockerfile

(streamlit_app/Dockerfile.streamlit)

```
FROM python:3.11-slim

WORKDIR /app

ENV STREAMLIT_SERVER_HEADLESS=true
ENV STREAMLIT_BROWSER_GATHER_USAGE_STATS=false

COPY requirements.streamlit.txt .
RUN pip install --no-cache-dir -r requirements.streamlit.txt

COPY . .

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0"]
```

Streamlit Requirements

(requirements.streamlit.txt)

```
streamlit
requests
```

Docker Compose (docker-compose.yml)

```
version: "3.9"

services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    ports:
```

```
- "8000:8000"

streamlit:
  build:
    context: ./streamlit_app
    dockerfile: Dockerfile.streamlit
  ports:
    - "8501:8501"
  depends_on:
    - backend
```

Streamlit → Backend Connection

Inside `streamlit_app/app.py`:

```
BACKEND_URL = "http://backend:8000"
```

This allows Streamlit to talk to the backend **inside Docker**.

Running Phase 5

Build & Run Everything

```
docker compose up --build
```

Access Applications

Service	URL
Streamlit UI	http://localhost:8501
FastAPI Docs	http://localhost:8000/docs